NLP

19-06-26

1 lac articles $\xrightarrow{\text{chunking}}$ 15 lac paragraph chunks

find the articles related to 'impact of trees on environment'

↓

query text

1) Get a trained NLP model
   - trained on any NLP task on any data

2) Generate sentence embeddings of all 15 lac chunks using NLP model

3) Generate sentence embedding of query text

4) Compute cosine similarities between query embedding and all 15 lac chunk embeddings

5) Sort in descending order of cosine similarity and select top K most similar chunks.

↓

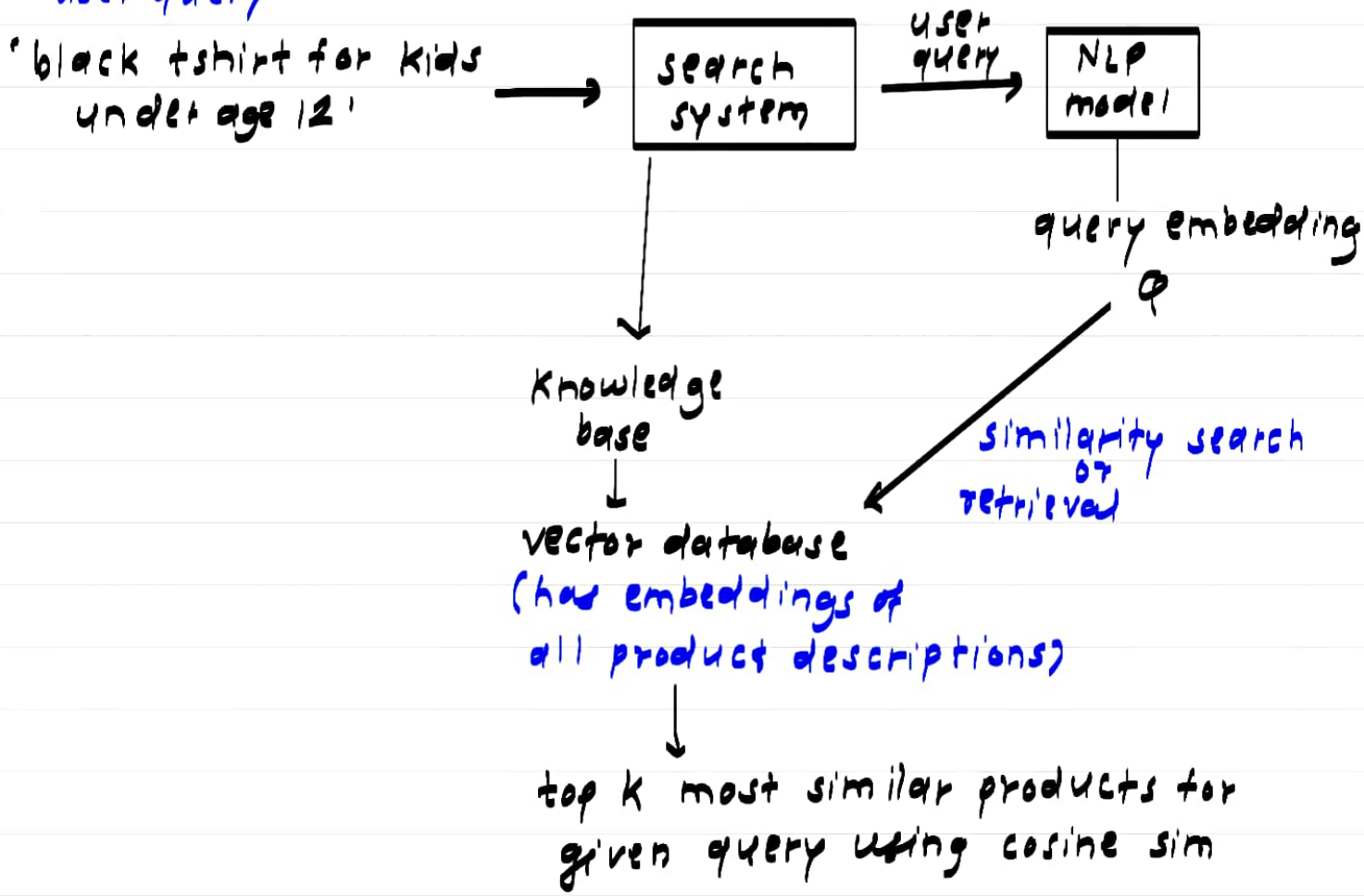
K - decided based on use case.

Retrieval → finding most similar content from a large pool of contents.

Retrieval + LLM Que-Ans system → Retrieval Augmented Generation - RAG

Working example of retrieval:

1) product search on e-commerce:



2) RAG

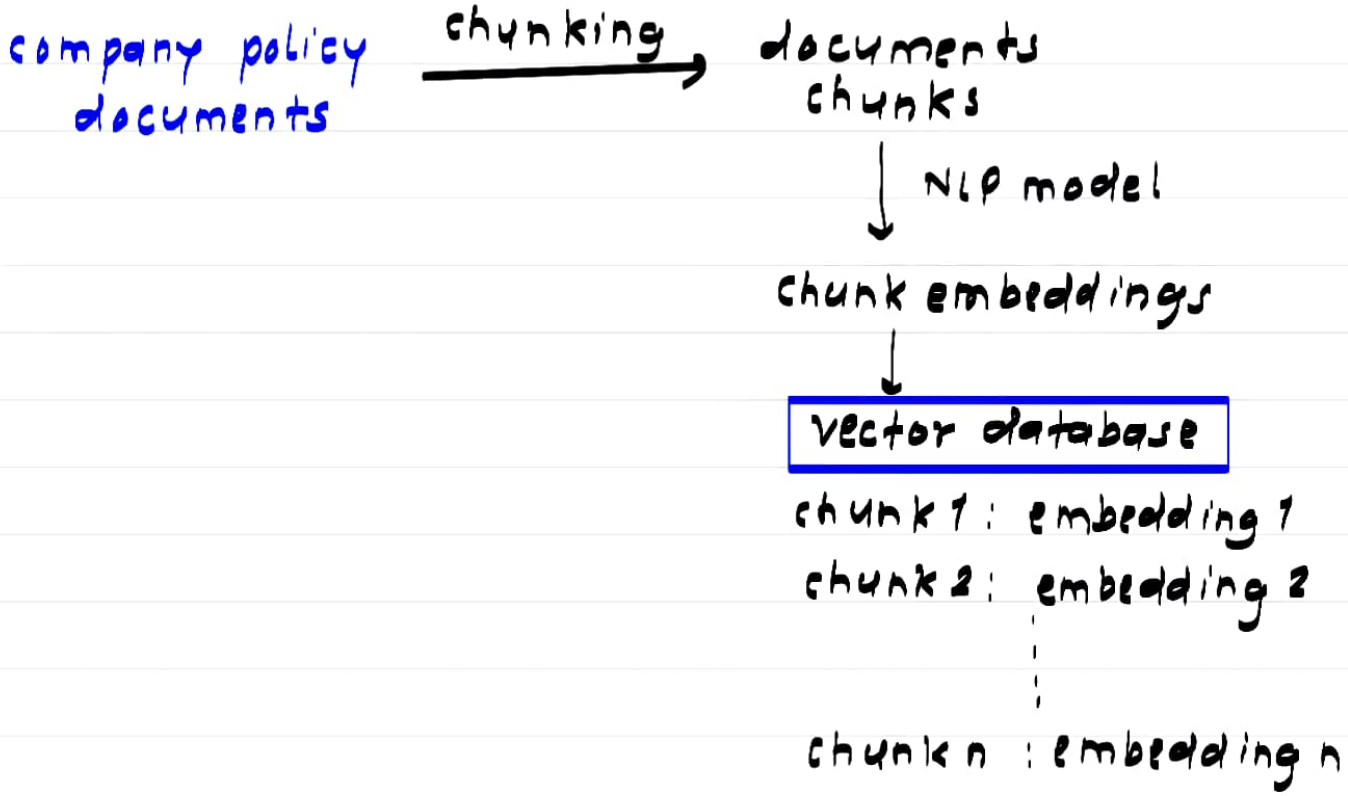
user query

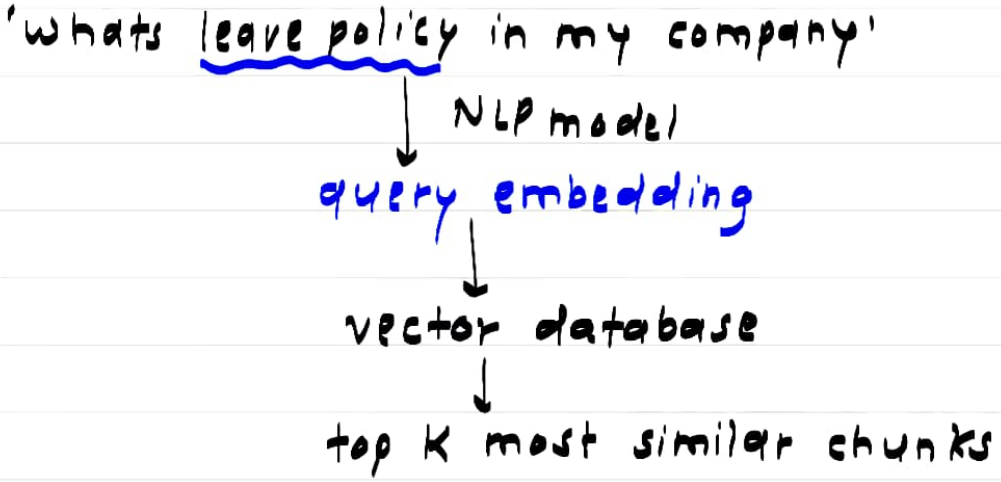
whats the leave policy of my company?

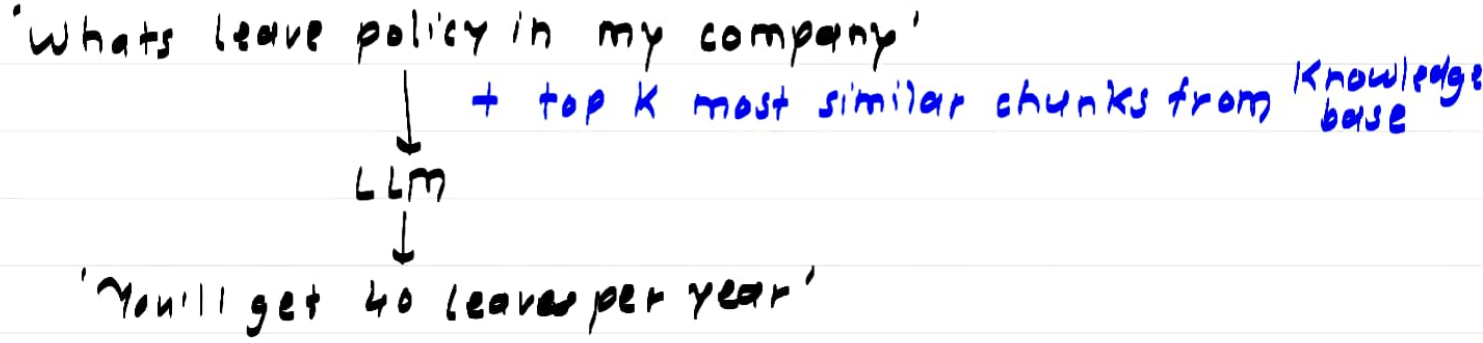
LLM doesn't know this answer
it searched on web and still didn't find the answer

for such systems for which LLM don't have any knowledge, we build RAG systems

- We'll create the internal company specific knowledge base.







Transfer learning in NLP:

eg. We need to train a sentiment Analysis model for our application reviews on play store with 2k customer reviews

text	label
t1	+ve
t2	-ve
⋮	neutral
	⋮
t2k	+ve

2k reviews is very small for such task with this data, model won't be able to learn fine details of the language and will fail in longer run.

fine details of language ⟶ Grammer, sentence structure, indent etc

In such cases we use transfer learning

We take a pretrained NLP model which already understands fine details of language as its already trained on huge corpus of text.

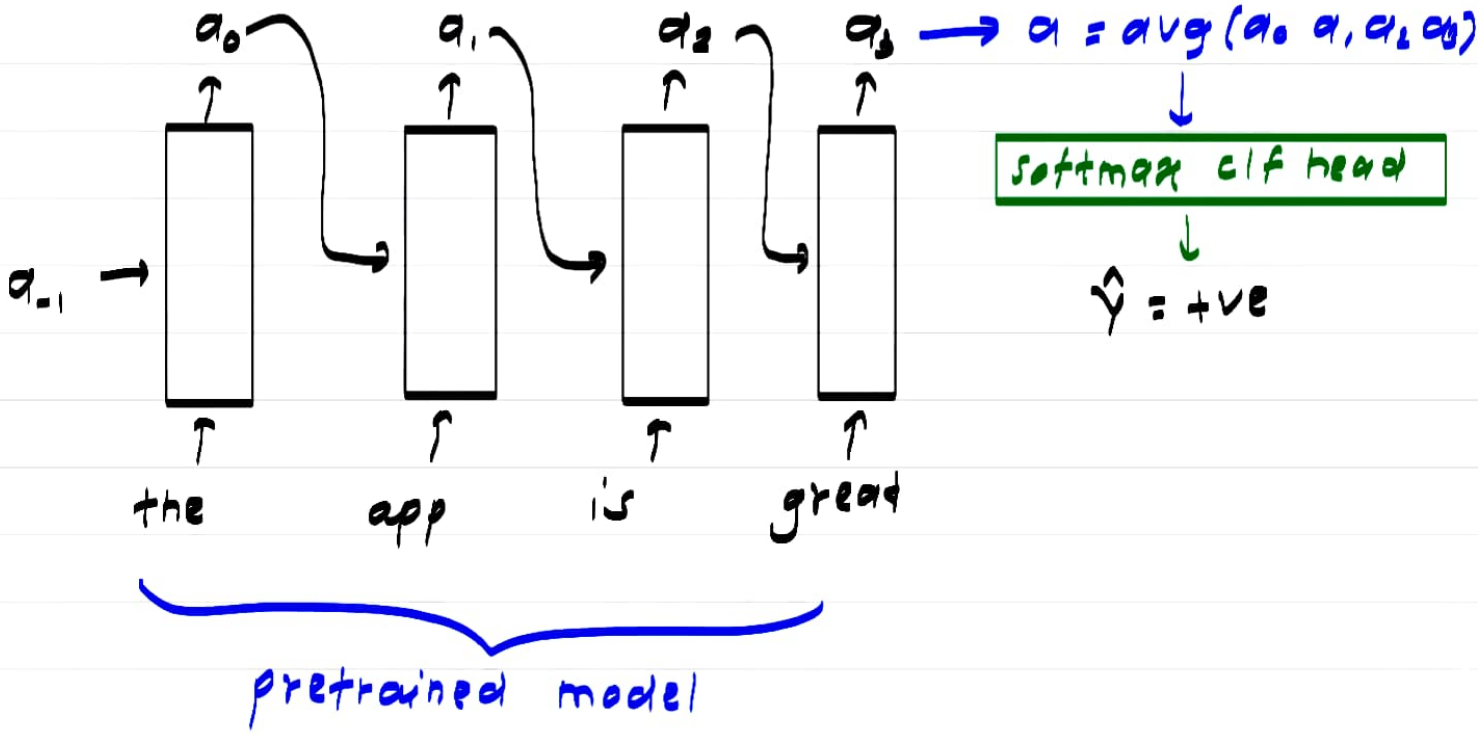
We only do the 'use case specific' fine tuning of this model using our custom dataset.

Model m1 → trained on 10M+ articles data for text classification

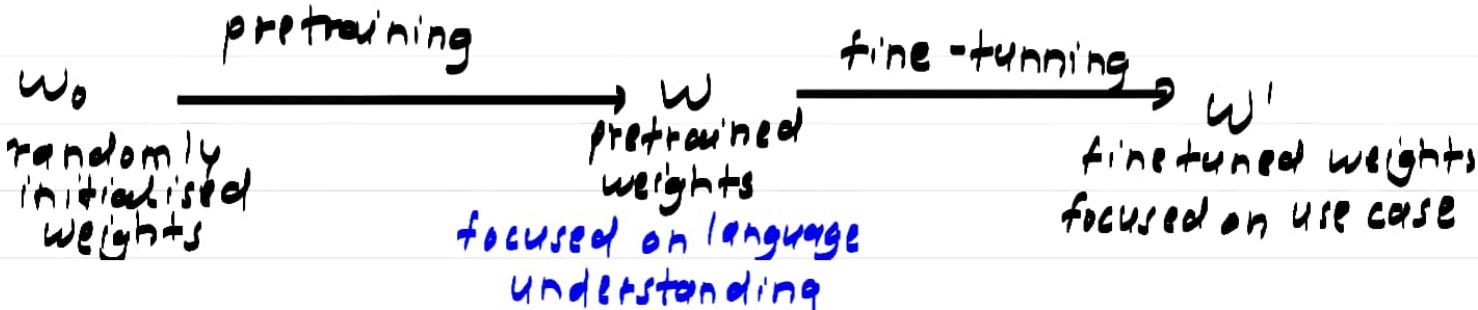
- sports
- environment
- Tech
- politics
- etc

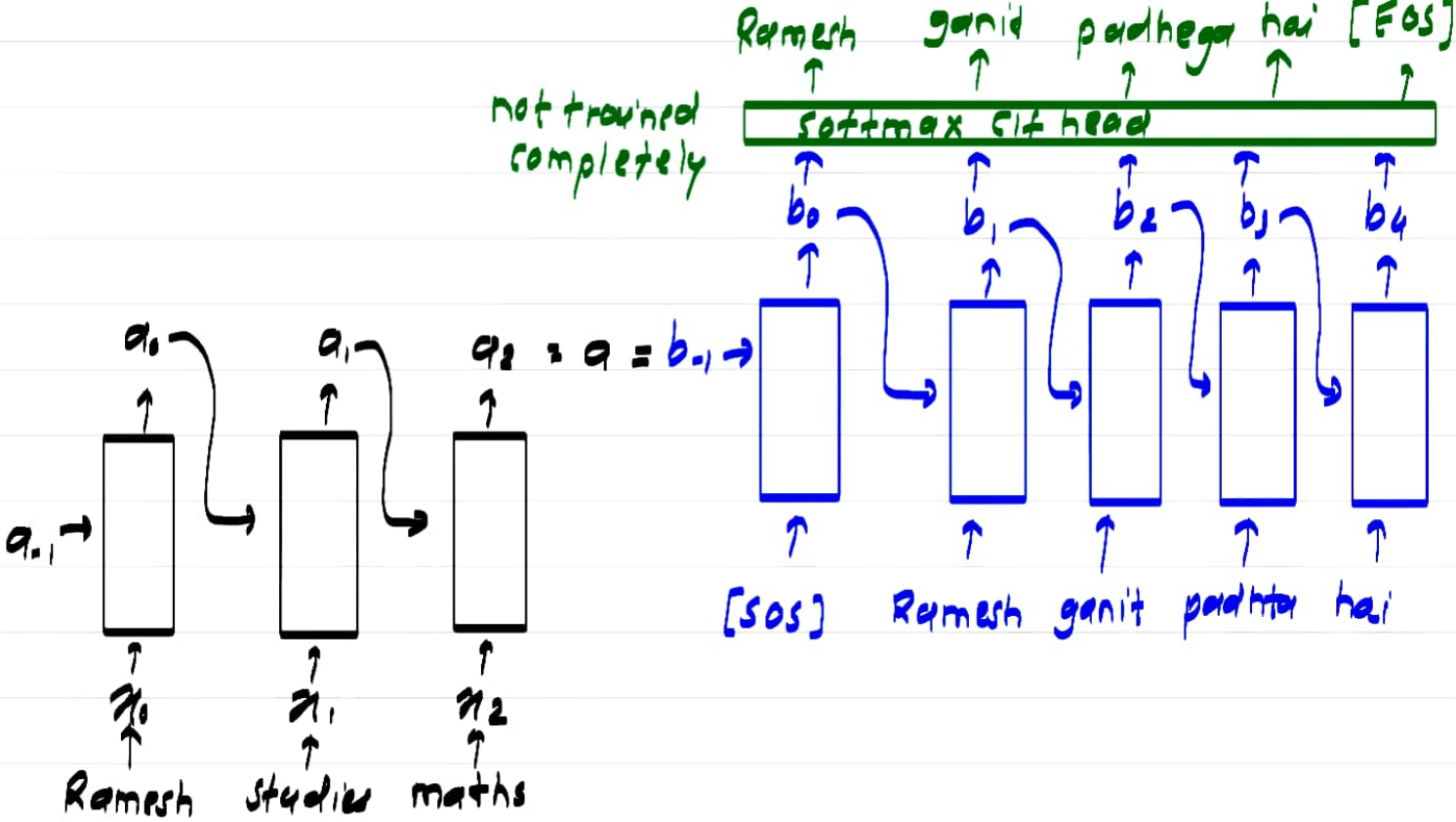
- grammer
- intent
- english vocab
- context
- etc

Now we'll use transfer learning and leverage the knowledge of this model to solve our use case.

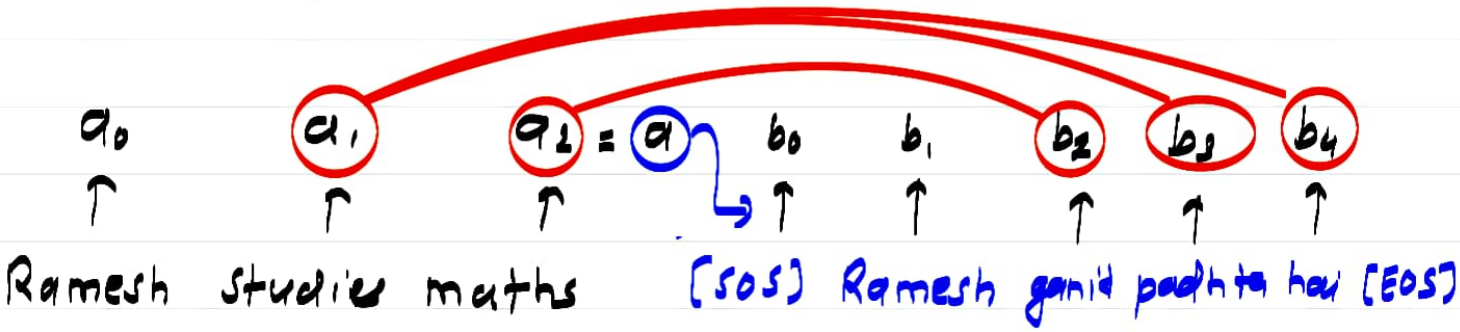


w - model parameters are updated during fine-tuning





As sequence length increases beyond few hundred tokens, performance of RNN / LSTM degrades drastically



Instead on just passing the average of all vectors from encoder to decoder, it would be better to pass all vectors from encoder to decoder

more vectors → more information → more context → better translation